

Garis Besar Cara Kerja Program bola5.cpp

1. Struktur Data dan Inisialisasi Awal

Di bagian awal, program mendefinisikan sebuah struktur data bernama `CetakanBola` untuk menyimpan informasi penting tentang setiap bola:

- `x` dan `y`: Menyimpan posisi koordinat bola saat ini.
- `dx` dan `dy`: Menyimpan arah dan kecepatan pergerakan bola ($d = \text{delta}$ atau selisih).

Logika Inisialisasi Posisi:

Di dalam fungsi `main`, program membuat ruang berukuran 40x20 dan membuat array berisi 5 bola. Posisi awal bola ditentukan menggunakan rumus berikut:

- `bola[i].x = (width / 2) + (i * 2);`
- `bola[i].y = (height / 2) - i;`

Penjelasan Rumus: Rumus ini bertujuan meletakkan bola di tengah ruangan (`width / 2` dan `height / 2`), lalu memberikan jarak antar bola. Nilai `+(i * 2)` menggeser koordinat X setiap bola sejauh 2 spasi ke kanan dari bola sebelumnya, dan `- i` menggeser koordinat Y satu baris ke atas. Ini dilakukan agar kelima bola tidak bertumpuk di satu titik saat program pertama kali dijalankan. Kecepatan awal semua bola diset berjalan ke arah kanan bawah (`dx = 1, dy = 1`).

2. Penggambaran Layar (Rendering)

Program menggunakan *infinite loop* (`while (true)`) untuk membuat animasi. Setiap *frame* animasi dimulai dengan membersihkan layar menggunakan `system("cls")`.

Setelah itu, program menggambar layar baris demi baris dan kolom demi kolom menggunakan perulangan bersarang (*nested loop* baris `i` dan kolom `j`):

1. **Mendeteksi Bola:** Program akan mengecek ke dalam array kelima bola. Jika ada bola yang koordinat `x` dan `y`-nya sama persis dengan baris dan kolom yang sedang digambar, maka ia akan mencetak `'O'`.
2. **Menggambar Tembok:** Jika posisi yang sedang digambar berada di ujung paling atas, bawah, kiri, atau kanan (`i == 0 || i == height - 1 || j == 0 || j == width - 1`), program mencetak tembok `'#'`.
3. **Ruang Kosong:** Jika tidak ada bola dan bukan bagian tembok, program mencetak spasi kosong.

3. Logika Pergerakan dan Pantulan Tembok (Fisika Sederhana)

Setelah layar selesai digambar, program harus menghitung posisi bola untuk *frame* berikutnya.

Rumus Pergerakan:

`bola[k].x += bola[k].dx;`

`bola[k].y += bola[k].dy;`

Artinya, posisi bola saat ini ditambah dengan kecepatannya. Jika dx positif, bola bergerak ke kanan. Jika dx negatif, bola bergerak ke kiri. Hal yang sama berlaku untuk dy (atas/bawah).

Logika Pemantulan (Collision Detection):

Tembok bagian dalam ada di batas koordinat 1 dan width - 2 (untuk sumbu X) serta 1 dan height - 2 (untuk sumbu Y). Ketika bola menyentuh batas ini, arah pergerakannya harus dibalik.

Rumus Kecepatan Acak:

Program menggunakan rumus `(rand() % 2) + 1` saat bola memantul.

- `rand() % 2` akan menghasilkan nilai acak antara **0** atau **1**.
- Dengan ditambah **1**, hasil akhirnya selalu **1** atau **2**.

Artinya, setiap kali bola menabrak tembok, bola tersebut tidak memantul dengan kecepatan yang konstan, melainkan secara acak akan bergerak sejauh 1 atau 2 petak per *frame*. Jika menabrak tembok kanan atau bawah, rumus ini dikalikan negatif (misal: `-((rand() % 2) + 1)`) agar arahnya berbalik ke kiri atau atas.

4. Pengaturan Frame Rate (Kecepatan Animasi)

Di akhir perulangan, terdapat perintah `this_thread::sleep_for(chrono::milliseconds(50));`.

Perintah ini menghentikan sementara jalannya program selama 50 milidetik sebelum mengulang layar berikutnya. Tanpa kode ini, komputer akan memproses *looping* terlalu cepat sehingga pergerakan bola tidak akan terlihat oleh mata manusia (hanya terlihat berkedip sangat cepat).

Garis Besar Cara Kerja Program bola50.cpp

Secara keseluruhan program bola50 dengan bola5 memiliki dasar yang sama tetapi memiliki beberapa perbedaan seperti:

1. Penambahan Library dan Pengacakan (Random Seed)

- `#include <cmath>` **dan** `#include <ctime>`: Ditambahkan untuk mendukung fungsi matematika absolut (`abs()`) dan mengambil waktu sistem (`time(0)`).
- `srand(time(0));`: Ditambahkan di awal `main()`. Ini berfungsi sebagai *seed* agar angka acak yang dihasilkan oleh `rand()` benar-benar berbeda setiap kali program dijalankan (berdasarkan waktu komputer saat ini). Pada kode sebelumnya, pantulan acak akan selalu memiliki pola yang sama setiap program di-restart.

2. Skala Ruangan dan Jumlah Bola Membesar

- Ruangan diperbesar menjadi `width = 100` dan `height = 100`.
- Jumlah bola (`jumlahBola`) ditingkatkan drastis dari 5 menjadi 50 bola.

3. Posisi dan Arah Awal yang Sepenuhnya Acak

- Alih-alih menyusun bola secara rapi di tengah ruangan, kode ini meletakkan 50 bola secara acak di sembarang koordinat dalam ruangan: `(rand() % (width - 2)) + 1`.
- Arah gerak awal bola (`dx` dan `dy`) juga diacak antara **1** atau **-1** menggunakan operator *ternary* (`rand() % 2 == 0`) ? 1 : -1.

4. Optimasi Penggambaran Layar (Mencegah Flicker)

- **Penggunaan** string `layar = ""`; Pada kode sebelumnya, setiap karakter dicetak satu per satu langsung ke layar (`cout << 'O'`). Pada versi ini, program menampung seluruh susunan karakter frame ke dalam sebuah variabel teks (`layar += 'O'`).
- Setelah seluruh baris dan kolom selesai diproses, barulah keseluruhan *frame* dicetak sekaligus dengan `cout << layar`; Teknik ini mirip dengan *Double Buffering* dan sangat efektif untuk mencegah layar berkedip (*flicker*) saat mencetak area yang besar (100x100).

5. Fitur Baru: Tabrakan Antar Bola (Inter-ball Collision)

Ini adalah logika baru yang paling signifikan. Program sekarang mengecek apakah sebuah bola menabrak bola lainnya:

- **Pengecekan Jarak:** Menggunakan for bersarang untuk membandingkan posisi bola[i] dan bola[j]. Jika jarak mutlak sumbu X dan Y antara kedua bola kurang dari atau sama dengan 1 ($\text{abs}(\text{bola}[i].x - \text{bola}[j].x) \leq 1$), berarti mereka saling bersentuhan.
- **Tukar Kecepatan (Fisika Sederhana):** Jika bertabrakan, arah dan kecepatan gerak (dx dan dy) dari kedua bola tersebut saling ditukar menggunakan variabel bantuan (temp_dx dan temp_dy). Ini mensimulasikan hukum kekekalan momentum pada benda dengan massa yang sama (memantul satu sama lain).
- **Pemisahan Paksa:** Kode `bola[i].x += bola[i].dx`; langsung dijalankan saat itu juga agar bola saling menjauh sedikit, mencegah *bug* di mana dua bola terus-menerus menempel dan bertukar arah berulang kali di titik yang sama.

6. Kecepatan Animasi Diperlambat

- Jeda waktu (sleep_for) diubah dari 50 milidetik menjadi 300 milidetik. Animasi ini akan berjalan jauh lebih lambat (seperti gerak lambat / *slow-motion*) dibandingkan versi sebelumnya, kemungkinan untuk memberi waktu mata manusia melihat kalkulasi tabrakan antar 50 bola tersebut.